AMATH 582: CLASSIFYING RED WINE BY QUALITY

PARTHA KRISHNAMURTHY

Applied And Computational Mathematics, University of Washington, Seattle, WA
ipartha2@uw.edu

1. Introduction and Overview

Given a dataset of input-output pairs (x_i, y_i), the primary goal of Supervised Machine Learning algorithms is to learn the function that maps the inputs to outputs. In Regression algorithms y_i is a continuous variable, while y_i is categorical in Classification. Multiclass classification categorizes the data into multiple mutually exclusive categories. Kernel Ridge Regression Classifier is a massive leap in mathematical power. While Ridge regularization prevents overfitting by constraining the model's complexity, the kernel trick allows the model to capture non-linear patterns in the dataset. In this paper, I compare several kernel methods applied to a Ridge Classification framework to predict the quality of wine.

2. Theoretical Background

For a given training data set of input-output pairs $\{(x_i, y_i)\}_{i=1}^N$, the assumption of supervised learning is that there is an unknown function $f^\dagger$ such that

$$y_i = f^\dagger(x_i) + \epsilon_i$$

where ϵ_i is assumed to be i.i.d Gaussian. The goal of supervised learning is to find a function $\hat{f}$ such that $\hat{f} \approx f^\dagger(x)$ for new inputs x that are not in the training set.

2.1. Maximum Likelihood Estimator, Least Squares and Regression.

The function $\hat{f}$ is chosen as the function within the class $\mathcal{F}$ that maximizes the likelihood of the observed data. The likelihood can be denoted as

$$\pi(y|f) = \prod_{i=1}^N \pi(y_i|x_i) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(f(x_i) - y_i)^2}{2\sigma^2}\right)$$

Given the likelihood is a probability distribution, it is more convenient to minimize its negative logarithm, given by:

$$-\log \pi(y|f) = \frac{1}{2\sigma^2} \sum_{i=1}^N (f(x_i) - y_i)^2 + \frac{N}{2} \log(2\pi\sigma^2)$$

Given the second term is not dependent on f, minimizing the negative log-likelihood is equivalent to minimizing the sum of squared errors:

$$L(f) \equiv L(y|f) := \sum_{i=1}^N (f(x_i) - y_i)^2$$

This gives a formal definition for $\hat{f}$ as

$$\hat{f} = \arg\min_{f \in \mathcal{F}} L(f)$$

where $\hat{f}$ is often called the **Maximum Likelihood Estimator (MLE)** of $f^\dagger$ within the class $\mathcal{F}$.

Date: 02/28/2026.

The model class, $\mathcal{F}$, can be expressed as a linear combination of fixed basis functions:

$$\mathcal{F} = \left\{ f(x) = \sum_{j=1}^{p} \theta_j \psi_j(x) : \theta_j \in \mathbb{R} \right\}$$

where ψ_j are the basis functions and θ_j are the coefficients. This reduces the MLE problem to finding the vector θ that minimizes the sum of squared errors:

$$\hat{\theta} = \arg\min_{\theta \in \mathbb{R}^p} \sum_{i=1}^{N} \left(\sum_{j=1}^{p} \theta_j \psi_j(x_i) - y_i \right)^2$$

The above problem can be expressed in matrix form as:

$$\hat{\theta} = \arg\min_{\theta \in \mathbb{R}^p} \|\Psi\theta - y\|_2^2$$

where $\Psi \in \mathbb{R}^{N \times p}$ is called the design matrix.

The closed-form solution to the above problem is given by the normal equation:

$$\hat{\theta} = (\Psi^T \Psi)^{-1} \Psi^T y \tag{1}$$

2.1.1. *Classification.* **Classification** and *Regression* are both fundamental subcategories of Supervised Learning. While regression predicts continuous variables, the output variable y_i in classification is categorical. For a binary training dataset $\{(x_i, y_i)\}_{i=1}^N$, the output values are typically defined as $y_i \in \{-1, +1\}$. The goal of classification is to learn a function f such that for every new input x, evaluating $f(x)$ indicates the class to which x belongs.

In the Least Squares Classification approach, this binary problem can be formulated exactly like a *regression* problem:

$$y_i = f^\dagger(x_i) + \epsilon_i$$

Then $\hat{f}$ is found by solving the least squares problem:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{N} (f(x_i) - y_i)^2$$

For any new data point, x, the class label can be predicted by looking at the sign of $\hat{f}(x)$.

The above formulation defines binary classification where the output has *two* categories. For multi-class classification problems, the standard practice is to encode the labels as one-hot vectors. If we have K classes, we encode the class labels as vectors in $\mathbb{R}^K$ where the k-th class is represented by the vector:

$$y_i = [0, \ldots, 0, 1, 0, \ldots, 0]^T = e_k$$

where the 1 is exclusively in the k-th position.

2.2. **Regularization.** The bias-variance tradeoff is a fundamental concept in Machine Learning that describes the bias and variance of a model and its prediction error.

- **Bias** refers to the error introduced by the approximation by a simpler model. A model paying less attention to training data will oversimplify the model leading to systematic errors in prediction.
- **Variance** refers to the error introduced by the model's sensitivity to small fluctuations in training data. The model pays too much attention to the data and captures noise leading to overfitting. These models perform poorly on unseen data.

2.2.1. *Controlling complexity with Ridge Regularization.* Regularization is a method of adding a penalty term to the loss function. This encourages simpler models and avoids overfitting. The regularized estimator $\hat{f}_\lambda$ is given by

$$\hat{f}_\lambda = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{N} (y_i - \theta^T \Psi(x_i))^2 + \lambda \|\theta\|_2^2$$

where λ is the regularization parameter that controls the strength of the penalty.

The regularized coefficient $\hat{\theta}_\lambda$ for the direct form solution described in 1 can then be modified as

$$\hat{\theta}_\lambda = (\Psi^T \Psi + \lambda I)^{-1} \Psi^T y \tag{2}$$

where I is the identity matrix of appropriate dimensions.

2.3. **Kernel Methods.** Given a set $\mathcal{X}$, a kernel K is a function that maps $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$. K is positive definite and symmetric (PDS) if the following is true:

- *Symmetry:* $K(x, y) = K(y, x) \forall x, y \in \mathcal{X}$
- *Positive Definiteness:* For any $n \in \mathbb{Z}_+$, any $x_1, \ldots, x_n \in \mathcal{X}$, and any $\zeta_1, \ldots, \zeta_n \in \mathbb{R}$,

$$\zeta^T K \zeta = \sum_{i=1}^{n} \sum_{j=1}^{n} \zeta_i \zeta_j K(x_i, x_j) \geq 0 \tag{3}$$

2.3.1. *Kernel Ridge Regression.* Kernel Ridge Regression (KRR) extends standard Ridge Regression by applying the "Kernel Trick," allowing the model to capture non-linear relationships without explicitly computing transformations in high-dimensional space.

By the Representer Theorem, the optimal regularized function can be expressed as a linear combination of the kernel evaluations at the training points:

$$f(x) = \sum_{i=1}^{N} \alpha_i K(x, x_i)$$

where $\alpha \in \mathbb{R}^N$ represents the dual weights assigned to each training sample, rather than weights assigned to individual features.

Substituting this into the Ridge Regression objective shifts the optimization from the primal formulation to the dual formulation. The closed-form solution for the optimal weights α becomes:

$$\alpha = (K + \lambda I)^{-1} y$$

where $K \in \mathbb{R}^{N \times N}$ is the Gram matrix with entries $K_{ij} = K(x_i, x_j)$, λ is the regularization parameter, and I is the identity matrix.

To classify a new unseen data point x_{test}, the model computes its similarity to all training points and weights them by α:

$$\hat{y} = \sum_{i=1}^{N} \alpha_i K(x_{test}, x_i)$$

In a multi-class setting, this process is repeated for each one-hot encoded class, and the final predicted class is the one that yields the maximum value.

2.4. **Cross Validation.** The optimal choice of model and parameter tuning is critical to achieving maximum success when the model predicts labels for unseen data. One of the approaches that is widely used is **Cross Validation**. The training data is split into multiple subsets, called *folds*. The model is trained on all but one of the folds and evaluated on the remaining validation fold. The process is repeated for different combinations of training and validation folds to pick the model parameters that yield the best average performance.

3. Algorithm Implementation and Development

This section will list the core algorithms that were used in the classification of wine by quality. A subset of wine quality data that is available on the UCI Machine Learning repository was used to test the performance of the models.

This section will cover the following algorithms:

- Data loading and pre-processing
- One-hot encoding of labels
- Linear Classifier
- Kernel methods
- Kernel Ridge Regression Classifier
- k-fold Cross Validation

3.1. Data loading and Pre-Processing.

One of the critical steps when training Machine Learning models is to ensure a clean dataset. A basic analysis of the data helps in understanding high-level patterns in the data.

Algorithm 1 Data loading and pre-processing

Require: Training data file train-data.csv, Testing data file test-data.csv
Ensure: $train_X \in \mathbb{R}^{N\times n}$, $train_Y \in \mathbb{R}^{N\times 1}$, $test_X \in \mathbb{R}^{m\times n}$, $test_Y \in \mathbb{R}^{m\times 1}$

1: Load the training data file, train ← LOADTXT(train-data.csv)
2: train_X, train_Y ← train[:, :-1], train[:, -1, None]
3: Load the testing data file, test ← LOADTXT(test-data.csv)
4: test_X, test_Y ← test[:, :-1], test[:, -1, None]
5: Standardize features: train_X ← $\frac{train_X-\mu_{train}}{\sigma_{train}}$, test_X ← $\frac{test_X-\mu_{train}}{\sigma_{train}}$
6: unique_values, counts ← UNIQUE(train_Y[:, 0])
7: PRINT(zip(unique_values, counts)) ▷ Obtain count of unique labels and display them
8: **return** train_X, train_Y, test_X, test_Y

The above algorithm parsed the files, normalized the feature scales to ensure reliable distance metrics for kernel evaluations, and returned the dataset in a matrix format, which is the desired format for running machine learning algorithms. The analysis performed on the dataset revealed an uneven distribution of wine quality where a couple of labels were favored heavily over the rest.

3.2. One-Hot Encoding of Labels.

This is a simple yet powerful technique to ensure the model does not assume an unwarranted ordinal relationship or continuous scale between the categorical labels.

Algorithm 2 One-Hot Encoding of Labels

Require: Label vector $y \in \mathbb{R}^{N\times 1}$, total number of classes k
Ensure: One-hot encoded labels $one_hot_y \in \mathbb{R}^{N\times k}$

1: $N = y.shape[0]$ ▷ Obtain the number of labels
2: Initialize a matrix of zeros, one_hot_y = ZEROS$((N, k))$
3: $one_hot_y[\text{ARANGE}(N), y.flatten()] = 1$ ▷ Vectorized assignment to set 1 at the target class index across all rows
4: **return** one_hot_y

3.3. Linear Classifier.

The linear classifier solves the normal equation for minimizing the least squares problem. This section will describe the normal equation solver and the classifier algorithm that invokes the normal equation solver.

Algorithm 3 Normal Equation Solver

Require: Data $X \in \mathbb{R}^{N\times n}$, One-hot encoded labels $Y \in \mathbb{R}^{N\times k}$
Ensure: Weight matrix $W \in \mathbb{R}^{n\times k}$
1: xt_x = MATRIX_MULTIPLICATION(X^T, X)
2: xt_y = MATRIX_MULTIPLICATION(X^T, Y)
3: xt_x_inv = INVERSE(xt_x)
4: W = MATRIX_MULTIPLICATION(xt_x_inv, xt_y)
5: **return** W

Algorithm 4 Linear Classifier

Require: Training data $train_X \in \mathbb{R}^{N\times n}$, One-hot training labels $train_Y \in \mathbb{R}^{N\times k}$, Test data $test_X \in \mathbb{R}^{m\times n}$, True test labels $test_Y \in \mathbb{R}^{m\times 1}$
Ensure: Percent of successful label prediction of test dataset, $success_percent$
1: $train_X$ = HSTACK$(\mathbf{1}, train_X)$ ▷ Append a column of 1s to incorporate the bias (intercept) term
2: $test_X$ = HSTACK$(\mathbf{1}, test_X)$ ▷ Append a column of 1s to incorporate the bias (intercept) term
3: Obtain the weights by calling the normal equation solver 3 using training data and labels, W = NORMAL-EQ-SOLVER$(train_X, train_Y)$
4: Get the raw prediction scores for test data, z = MATRIX_MULTIPLICATION$(test_X, W)$
5: Get the predicted label by finding the index of the maximum score, y_pred = ARGMAX(z)
6: Calculate model performance as success percentage, $success_percent$ = SUM$(y_pred == test_Y.flatten())/m$
7: **return** $success_percent$

3.4. **Kernel methods.** This section will define the mathematical formula and the corresponding pseudo code for the kernel methods that were used in the classifier.

3.4.1. *RBF Kernel.*

$$k_{RBF}(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right) \tag{4}$$

Algorithm 5 RBF Kernel

Require: The two sets of points to evaluate the kernel X, X', Gaussian width σ
Ensure: Kernel, $k_{RBF} \in \mathbb{R}^{N\times N}$
1: Obtain the difference for each pair of data points, $diff = X[:, None, :] - X'[None, :, :]$ ▷ Vectorization method by broadcasting different indices and subtracting every pair of points
2: Calculate the squared norm of the differences, $norm = (\text{NORM}(diff))^2$
3: Calculate Kernel, k_{RBF} = EXPONENTIAL$(-\frac{norm}{2*\sigma^2})$
4: **return** k_{RBF}

3.4.2. *Laplace Kernel.*

$$k_{laplace}(x, x') = \exp\left(-\frac{\|x - x'\|_1}{\sigma}\right) \tag{5}$$

3.4.3. *Polynomial Kernel.*

$$k_{poly}(x, x') = (x^T x' + c)^d \tag{6}$$

Algorithm 6 Laplace Kernel

Require: The two sets of points to evaluate the kernel X, X', Gaussian width σ
Ensure: Kernel, $k_{laplace} \in \mathbb{R}^{N \times N}$
1: Obtain the difference for each pair of data points, $diff = X[:, None, :] - X'[None, :, :]$ ▷ Vectorization method by broadcasting different indices and subtracting every pair of points
2: Calculate the 1-norm of the differences, $norm = (\text{NORM}(diff, ord = 1))$
3: Calculate Kernel, $k_{laplace} = \text{EXPONENTIAL}(-\frac{norm}{\sigma})$
4: **return** $k_{laplace}$

Algorithm 7 Polynomial Kernel

Require: The two sets of points to evaluate the kernel X, X', degree d, constant c
Ensure: Kernel, $k_{poly} \in \mathbb{R}^{N \times N}$
1: Calculate the polynomial value, $poly = \text{MATRIX_MULTIPLICATION}(X, X'^T) + c$
2: Calculate kernel, $k_{poly} = poly^d$
3: **return** k_{poly}

3.4.4. *Kernel Ridge Regression solver.* This algorithm solves the Ridge regression by solving the normal equation for a specified kernel.

Algorithm 8 Kernel Ridge Regression solver

Require: The two sets of points to evaluate the kernel X, X', one-hot encoded labels one_hot_y, regression parameter λ, kernel equation solver method $kernel_solver$, and other kernel-specific configuration in a map $params$
Ensure: Weight of data points, α
1: Calculate kernel, $K = \text{KERNEL_SOLVER}(X, X', params)$ ▷ The kernel_solver could be any of the algorithms described in 5, 6, 7
2: Calculate inverse of regularized kernel, $k_inv_reg = \text{INVERSE}(K + \lambda I)$ ▷ I is the identity matrix of appropriate dimensions
3: Calculate weight of data points, $\alpha = \text{MATRIX_MULTIPLICATION}(k_inv_reg, one_hot_y)$
4: **return** α

3.5. **Kernel Ridge Regression Classifier.** This is the executor algorithm that invokes the Kernel Ridge Regression Solver 8 with one of the kernel methods listed in 5, 6, 7. This function returns the success percentage on the test data.

3.6. ***k*-fold Cross Validation.** This algorithm evaluates the robustness of a specific hyperparameter configuration by partitioning the training data into k disjoint subsets. It systematically trains the model on $k - 1$ folds and validates it on the remaining fold, ensuring every data point is used for validation exactly once to prevent test-set leakage.

4. Computational Results

4.1. **Basic Data Analysis.** The given training dataset was analyzed to study high-level patterns in the data. The data revealed severe class imbalance, with *two* labels accounting for 82% of the data. The distribution of labels is shown in Figure 1.

Algorithm 9 Kernel Ridge Regression Classifier

Require: Training data $train_X \in \mathbb{R}^{N\times n}$, True training labels $train_Y \in \mathbb{R}^{N\times 1}$, Test data $test_X \in \mathbb{R}^{m\times n}$, True test labels $test_Y \in \mathbb{R}^{m\times 1}$, kernel-specific configuration in a map $params$

Ensure: Percent of successful label prediction of test dataset, $success_percent$

1: One-hot encode the label Y, $one_hot_y = \text{ONE_HOT_ENCODE}(train_Y)$
2: Calculate weight of data points for specified kernel, $\alpha = \text{KRRSOLVER}(train_X, train_X, one_hot_y, params)$ ▷ KRRSolver refers to **??**
3: Calculate kernel for test data, $K_{test} = \text{KERNELCALCULATOR}(test_X, train_X, params)$ ▷ KernelCalculator is one of the kernel methods listed in 5, 6, 7.
4: Get the raw prediction scores for test data, $z = \text{MATRIX_MULTIPLICATION}(K_{test}, \alpha)$
5: Get the predicted label by finding the index of the maximum score, $y_pred = \text{ARGMAX}(z)$
6: Calculate model performance as success percentage, $success_percent = \text{SUM}(y_pred == test_Y.flatten())/m$
7: **return** $success_percent$

Algorithm 10 k-fold Cross Validation

Require: Training data $X \in \mathbb{R}^{N\times n}$, True training labels $Y \in \mathbb{R}^{N\times 1}$, Number of folds k, Kernel-specific configuration $params$

Ensure: Mean validation success percentage, $mean_score$

Shuffle the indices of the dataset to ensure a random distribution of classes
Partition X and Y into k equal-sized disjoint folds: $\{X_1, X_2, \ldots, X_k\}$ and $\{Y_1, Y_2, \ldots, Y_k\}$
Initialize an empty array to track fold performance, $scores = [\,]$
for $i = 1$ to k **do**
    Set validation data $X_{val} = X_i$ and validation labels $Y_{val} = Y_i$
    Set training data $X_{train} = \bigcup_{j\neq i} X_j$ and training labels $Y_{train} = \bigcup_{j\neq i} Y_j$
    $one_hot_y_train = \text{ONE_HOT_ENCODE}(Y_{train})$
    $\alpha = \text{KRRSOLVER}(X_{train}, X_{train}, one_hot_y_train, params)$
    $K_{val} = \text{KERNELCALCULATOR}(X_{val}, X_{train}, params)$
    $z = \text{MATRIX_MULTIPLICATION}(K_{val}, \alpha)$
    $y_pred = \text{ARGMAX}(z)$
    $fold_score = \text{SUM}(y_pred == Y_{val}.flatten())/|Y_{val}|$
    Append $fold_score$ to $scores$
end for
Calculate the average accuracy across all folds, $mean_score = \text{MEAN}(scores)$
return $mean_score$

4.2. **Linear Classifier Results.** An unregularized linear classifier struggles to make meaningful predictions given the severe class imbalance shown in Figure 1. Because the model minimizes the global sum of squared errors, the heavily represented majority classes completely dominate the loss function. Consequently, the decision boundary is aggressively skewed to fit these dense regions, effectively treating the minority classes as negligible statistical noise. As a result, the model was only able to achieve an accuracy of 52% on the test dataset.

4.3. **Comparing Kernel Methods.** With an optimal choice of hyperparameters, all *three* kernel methods significantly outperformed the linear classifier. The linear model failed because it attempts to slice the data with a flat hyperplane, whereas wine quality is determined by complex, non-linear chemical interactions. By utilizing the kernel trick, the Ridge Classifier successfully implicitly mapped these features into a high-dimensional space to capture those exact non-linear boundaries.

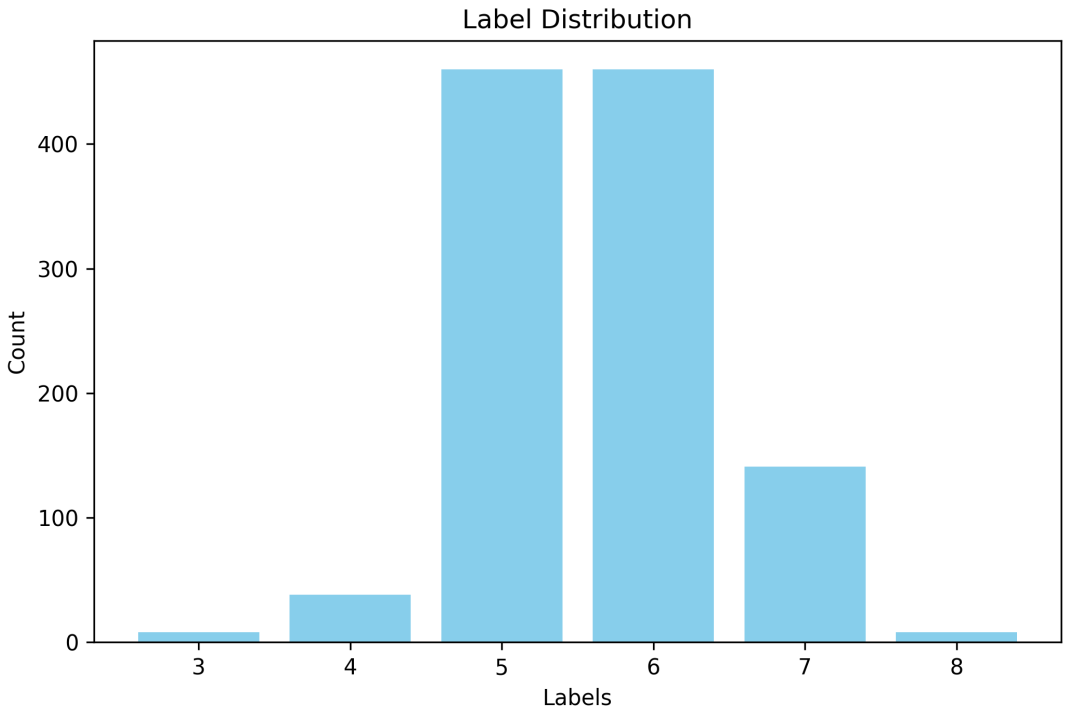


FIGURE 1. Labels (5) and (6) account for 82% of the dataset.

Across several combinations of hyperparameters during k-fold cross-validation, an accuracy of nearly 67% was achieved using the **Laplace** kernel. The Laplace kernel systematically outperformed the RBF kernel because it relies on the L1 (Manhattan) norm rather than the squared L2 (Euclidean) norm. This makes Laplace inherently more robust to outlier spikes in individual chemical features, which heavily distort the squared distances in RBF. The Polynomial kernel performed the poorest of the three because it applies a global mathematical curve, whereas the data required the localized "similarity" matching provided by the radial kernels.

Furthermore, the hyperparameter tuning revealed the geometric nature of the dataset. The optimal bandwidth parameter ($\sigma = 1$) for both RBF and Laplace was highly constrained, indicating that a wine must be chemically very close to a training sample to reliably share its quality label. Simultaneously, the regularization parameter (λ) dictated how aggressively the model filtered out the high-frequency noise of subjective human taste ratings, preventing the models from overfitting to overlapping class boundaries. The best performance of each kernel method, along with its optimal hyperparameters, is tabulated in Table 1.

Kernel Method	**Hyperparameters**	**CV Accuracy %**	**Test data accuracy %**
RBF	$\lambda = 1, \sigma = 1$	65.02	60.33
Laplace	$\lambda = 10, \sigma = 1$	67.35	64.10
Polynomial	$\lambda = 10, c = 1, d = 2$	60.45	60.12

TABLE 1. Kernel method performance

4.4. **Predictions on Unseen Data.** To validate the real-world applicability of the models, the trained classifiers were deployed to predict the quality labels for a new, previously unseen batch of wine samples.

Crucially, the raw features of this new batch were first standardized using the training dataset's mean and standard deviation to ensure mathematically valid kernel distance calculations.

The RBF and Laplace predictions perfectly matched, while the Polynomial kernel predicted a different class for certain samples. The results of the predictions are tabulated in Table 2.

Kernel Method	**Sample 1**	**Sample 2**	**Sample 3**	**Sample 4**	**Sample 5**
RBF	7	5	5	7	7
Laplace	7	5	5	7	7
Polynomial	6	4	5	6	6

TABLE 2. Quality prediction on new batch of wines

5. SUMMARY AND CONCLUSIONS

Kernel methods provide a powerful mathematical framework to learn non-linear interactions within a dataset. While Ridge regularization controls the complexity of the model by penalizing high weights to prevent overfitting, it is the application of the kernel trick that allows the classifier to successfully navigate datasets with severe class imbalance by projecting the features into a separable, high-dimensional space.

While RBF and Laplace are local similarity kernels due to their exponential decay with distance, the polynomial is a global kernel that amplifies feature alignment regardless of physical distance. The RBF kernel, utilizing Euclidean distance, is an ideal fit for rotationally invariant datasets or scenarios where anomaly detection is critical. Conversely, the Laplace kernel's reliance on Manhattan distance makes it highly robust to individual feature outliers, prioritizing the overall similarity of the combined feature set. While the polynomial kernel performed poorly in this specific use case, it has vast applications in fields like Natural Language Processing where the combined, synergistic interaction between features is more important than the individual features themselves.

k-fold Cross Validation remains the industry standard for tuning these complex hyperparameters. It ensures the model is rigorously built, tested, and validated before it ever evaluates unseen test data. Ultimately, this project highlights that machine learning is not a one-size-fits-all process; achieving optimal predictive performance requires a deep theoretical understanding of how different geometric distance metrics and kernel projections interact with the specific physical nature of the underlying data.